Lab Sheet 2 — Permissions, Processes and Package Management

Complete questions with answers immediately below each question. Commands are written for common Linux distributions; use sudo only when required.

Lab Sheet 2 — Permissions, Processes and Package

Management

distributions; use sudo only when required.

Lab Sheet 2 — Permissions, Processes and Package

Management

distributions; use sudo only when required.

Lab Sheet 2 — Permissions, Processes and Package

Management

sudo only when required.

Question 1: Create a file named notes.txt and display its permissions string using the ls -l command.

Answer / Solution:

```
touch notes.txt && ls -l notes.txt
```

Question 2: Change the permissions of notes.txt to 644 using numeric (octal) mode, and verify the changes using ls -l.

Answer / Solution:

```
chmod 644 notes.txt && ls -l notes.txt
```

Question 3: Change notes.txt permissions to 600 (owner read/write only, no permissions for group and others) and verify.

Answer / Solution:

```
chmod 600 notes.txt && ls -l notes.txt
```

Question 4: Create an executable file named script.sh and grant execute permission exclusively to the owner using symbolic mode.

Answer / Solution:

```
printf '#!/bin/bash\necho Hello' > script.sh && chmod u+x,go-x script.sh && ls -l script.sh
```

Question 5: Use the whoami command and ls -l notes.txt to identify the user and group ownership of notes.txt.

Answer / Solution:

```
whoami; ls -l notes.txt (the third field is owner; fourth is group)
```

Question 6: Run the ps command in your terminal and record how many processes are active in your current shell session.

Answer / Solution:

```
ps -f (count the process rows shown for this shell; output varies)
```

Question 7: Run ps aux and count or estimate the total number of running processes across the entire operating system.

Answer / Solution:

```
ps aux | wc -l (subtract the header line for an approximate count)
```

Question 8: Start a background process using sleep 60 & and immediately run jobs to verify that it is active in the background.

Answer / Solution:

```
sleep 60 &; jobs -l
```

Question 9: Use the kill command with the Process ID (PID) to terminate the active sleep process created in this exercise.

Answer / Solution:

```
jobs -l; kill <PID> (use the PID shown by jobs -l / pgrep -n sleep)
```

Question 10: Search for an available software package named tree using your system's package manager without installing it.

Answer / Solution:

```
apt-cache search ^tree$ (Debian/Ubuntu) or dnf search tree (Fedora/RHEL)
```

Question 11: Display the current umask value of your terminal shell environment.

Answer / Solution:

```
umask
```

Question 12: Use symbolic chmod syntax to remove write permissions from the group on a file named document.txt.

Answer / Solution:

```
chmod g-w document.txt
```

Question 13: Bring a background job with Job ID 1 back to the foreground using the fg command.

Answer / Solution:

```
fg %1
```

Question 14: Send a background job with Job ID 1 to resume execution in the background using the bg command.

Answer / Solution:

```
bg %1
```

Question 15: Display the kernel information and system architecture using uname -a.

Answer / Solution:

```
uname -a
```

Question 16: Display a snapshot of all processes running for your user account using ps -u $USER.

Answer / Solution:

```
ps -u "$USER"
```

Question 17: Use pkill to terminate a test process running under the name sleep.

Answer / Solution:

```
pkill -x sleep (only use when it is safe to stop all sleep processes you own)
```

Question 18: Check the current repository package index updates on an Ubuntu/Debian system using sudo apt update.

Answer / Solution:

```
sudo apt update
```

Question 19: Display the list of currently installed packages on your system using apt list --installed (or dnf list installed).

Answer / Solution:

```
apt list --installed (Debian/Ubuntu) or dnf list installed (Fedora/RHEL)
```

Question 20: Use chown to change the owner of script.sh to yourself explicitly, and verify with ls -l.

Answer / Solution:

```
sudo chown "$USER" script.sh && ls -l script.sh
```

Medium Exercises (Q21–Q35)

Question 21: Set permissions on script.sh so that the owner has full access (rwx), the group has read and execute (r- x), and others have zero permissions — write the numeric command.

Answer / Solution:

```
chmod 750 script.sh
```

Question 22: Explain the exact difference in rights between running chmod 700 file and running chmod 007 file.

Answer / Solution:

```
chmod 700 file gives owner rwx and no rights to group/others; chmod 007 gives owner no rights, group no
rights, others
```

rwx.

Question 23: Use chown to change both the user owner and group owner of a file to student:student in a single command.

Answer / Solution:

sudo chown student:student file

Question 24: Launch the top interactive process monitor, press 'P' to sort processes by CPU utilization, and press 'q' to exit.

Answer / Solution:

```
top; press P; press q to quit
```

Question 25: Start two background sleep processes (sleep 100 & and sleep 200 &), list them with jobs, and bring job 1 to the foreground using fg.

Answer / Solution:

```
sleep 100 &; sleep 200 &; jobs; fg %1 (Ctrl+Z to suspend, then bg %1 if needed)
```

Question 26: Use pkill with a safe target process name to terminate all active instances of that command simultaneously.

Answer / Solution:

```
pkill -x sleep (verify the target first with pgrep -a sleep)
```

Question 27: Install the tree package using your system's package manager (apt or dnf), and run tree on your home directory.

Answer / Solution:

```
sudo apt update && sudo apt install tree && tree ~ (Fedora/RHEL: sudo dnf install tree)
```

Question 28: Remove the tree package after testing, and verify its uninstallation using the package manager list command.

Answer / Solution:

```
sudo apt remove tree && apt list --installed 2>/dev/null | grep -w tree (or dnf list installed tree)
```

Question 29: Check your default umask setting, create a new file named test_mask.txt, and analyze how umask influenced its permissions.

Answer / Solution:

```
umask; touch test_mask.txt; ls -l test_mask.txt. New file mode is typically 666 minus umask (execute
bits are not set on
regular files).
```

Question 30: Filter system processes to list only those belonging to your logged-in username using ps aux | grep $USER.

Answer / Solution:

```
ps aux | grep -F "$USER" (prefer ps -u "$USER" for exact user filtering)
```

Question 31: Change group ownership of a directory named project_dir to group devteam using the chgrp command with sudo.

Answer / Solution:

```
sudo chgrp -R devteam project_dir && ls -ld project_dir
```

Question 32: Launch a process with a lower scheduling priority using nice -n 10 sleep 300 & and confirm its priority using ps or top.

Answer / Solution:

```
nice -n 10 sleep 300 &; ps -o pid,ni,cmd -p $!
```

Question 33: Modify the priority of an already running process using the renice command by specifying its PID.

Answer / Solution:

```
renice 10 -p <PID>; ps -o pid,ni,cmd -p <PID>
```

Question 34: Search for the apache2 or nginx package in apt/dnf, install it, and check its system status using systemctl status.

Answer / Solution:

```
sudo apt update && apt-cache search apache2; sudo apt install apache2; systemctl status apache2 (or
nginx). Check
before installing on a production machine.
```

Question 35: Explain why executing apt install <package> without running apt update first on a fresh session might cause installation failure.

Hard / Challenge Exercises (Q36–Q50)

Answer / Solution:

```
apt update refreshes package indexes; stale/missing indexes can cause package-not-found or version
errors.
```

Hard / Challenge Exercises (Q36–Q50)

Question 36: Create a shell script file, grant execute permissions using symbolic mode (chmod +x), execute it with ./script.sh, and explain why execution fails if the shebang line (#!/bin/bash) is missing or invalid.

Answer / Solution:

```
printf "#!/bin/bash\necho Hello" > script.sh && chmod +x script.sh && ./script.sh
```

Question 37: Set up permissions for shared.txt so the owner has read/write, the group has read-only, and others have zero access — write out both symbolic AND numeric commands.

Answer / Solution:

```
chmod 640 shared.txt (symbolic: chmod u=rw,g=r,o= shared.txt)
```

Question 38: Start a background task with nice -n 15 sleep 300 &, locate its PID in top, and explain how the 'nice value' impacts CPU time allocation.

Answer / Solution:

```
nice -n 15 sleep 300 &; top (find PID; NI shows nice value). Higher nice values generally mean lower CPU
scheduling
priority.
```

Question 39: Debug Scenario: A student runs kill 1234 but the process refuses to terminate. What escalating command should be used next, and why is it considered a 'last resort'?

Answer / Solution:

```
kill -TERM 1234; if it still will not stop and you have verified the PID, use kill -KILL 1234 as last
resort because it cannot be
caught or cleaned up.
```

Question 40: Display the top 5 CPU-consuming processes currently active on the system using ps aux --sort=-%cpu | head -5.

Answer / Solution:

```
ps aux --sort=-%cpu | head -n 6
```

Question 41: Construct a single-line command using find and chmod to grant execute permissions (+x) to all .sh files in your working directory tree simultaneously.

Answer / Solution:

```
find . -type f -name '*.sh' -exec chmod +x {} +
```

Question 42: Explain the security risks associated with running chmod 777 on a sensitive system configuration file in production.

Answer / Solution:

```
chmod 777 allows everyone to read, modify and execute the file; attackers can alter trusted
configuration. Use least
privilege, e.g. 640 or 600 as appropriate.
```

Question 43: Install a package (e.g., tree), and use the low-level package tool (dpkg -L <pkg> or rpm -ql <pkg>) to list every file installed onto the filesystem by that package.

Answer / Solution:

```
sudo apt install tree && dpkg -L tree (RHEL/Fedora: rpm -ql tree)
```

Question 44: Conceptual Analysis: Explain the difference between process termination via SIGTERM (Signal 15) vs SIGKILL (Signal 9), and why applications should handle SIGTERM.

Answer / Solution:

```
SIGTERM (15) requests graceful termination and can be handled for cleanup; SIGKILL (9) forces immediate
termination
and cannot be caught or ignored.
```

Question 45: Create two files owned by different users (using sudo chown), then write a single find command that filters and lists files owned by a specific user.

Answer / Solution:

```
sudo chown user1 file1 && sudo chown user2 file2 && find . -type f -user user1 -print
```

Question 46: Set the Set User ID (SUID) bit on an executable binary custom_tool, write the chmod command, and explain its execution privilege implications.

Answer / Solution:

```
sudo chmod u+s custom_tool (numeric form: sudo chmod 4755 custom_tool). SUID runs the executable with
its owner UID;
use only trusted binaries.
```

Question 47: Set the Set Group ID (SGID) bit on a shared folder /shared_dir using octal mode (2775), and explain how newly created files inherit group ownership.

Answer / Solution:

```
sudo chmod 2775 /shared_dir; new files/subdirectories inherit the directory group on typical Linux
filesystems.
```

Question 48: Set the Sticky Bit on a directory /public_tmp using symbolic (chmod o+t) or octal (chmod 1777) mode, and explain why it prevents unauthorized file deletion.

Answer / Solution:

```
sudo chmod o+t /public_tmp (or sudo chmod 1777 /public_tmp); users cannot delete/rename other users'
files in the sticky
directory unless privileged or file owner.
```

Question 49: Demonstrate job management: Suspend a running command using Ctrl+Z, verify its stopped state with jobs, resume it in the background with bg, and then attach it with fg.

Answer / Solution:

```
Press Ctrl+Z; jobs; bg %1; fg %1
```

Question 50: Write a shell command pipeline that identifies the process consuming the most memory (%MEM), logs its PID and details to a file named top_memory.log, and gracefully requests its termination using SIGTERM.

Answer / Solution:

```
ps -eo pid,comm,%mem --sort=-%mem | head -n 2 | tee top_memory.log; PID=$(ps -eo pid,%mem --sort=-%mem |
awk
'NR==2{print $1}'); [ -n "$PID" ] && kill -TERM "$PID"
```